

IF2224 TEORI BAHASA FORMAL DAN OTOMATA

TUGAS BESAR

Milestone 2

(Kelompok TBFO)



Dipersiapkan oleh:

13524044	Narendra Dharma Wistara M
13524060	Steven Tan
13524090	Nashiruddin Akram
13524100	Arghawisesa Dwinanda Arham

**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2026**

Daftar Isi

Daftar Isi.....	1
Teori Singkat.....	1
Perancangan & Implementasi.....	3
Pengujian.....	4
Kesimpulan dan Saran.....	5
Lampiran.....	6
Referensi.....	7

Teori Singkat

A. Parser

Parser adalah komponen pada compiler atau interpreter yang bertugas menerima input berupa token-token hasil lexical analysis, lalu memeriksa apakah urutan token tersebut membentuk struktur program yang valid sesuai grammar bahasa Arion.

Pada milestone sebelumnya, source code telah diproses oleh lexer menjadi token. Oleh karena itu, parser tidak lagi membaca karakter mentah dari source code, melainkan membaca deretan token atau *token stream*.

Dalam bentuk lebih formal, jika token stream dinyatakan sebagai:

$$T = \langle t1, t2, t3, ..., tn \rangle$$

maka tugas parser adalah menentukan apakah deretan token tersebut dapat diterima oleh grammar bahasa Arion.

$$T \in L(G)$$

dengan G adalah grammar bahasa Arion dan $L(G)$ adalah himpunan seluruh program valid yang dapat dibentuk dari grammar tersebut.

Secara garis besar, parser melakukan langkah-langkah berikut.

1. Membaca token hasil lexer secara berurutan.
2. Mencocokkan token dengan aturan grammar.
3. Menentukan apakah struktur token valid atau tidak.
4. Membangun parse tree jika input valid.
5. Menghasilkan syntax error jika ditemukan token yang tidak sesuai.

Dengan demikian, parser berfungsi sebagai jembatan antara representasi linear berupa token dan representasi hierarkis berupa struktur program.

B. Parser Tree

Parse tree adalah struktur pohon yang merepresentasikan bagaimana sebuah input dapat diturunkan dari grammar. Setiap node non-terminal merepresentasikan aturan grammar, sedangkan node terminal merepresentasikan token aktual yang muncul pada source code.

Secara umum, aturan produksi grammar dapat ditulis sebagai:

$$A \rightarrow \alpha$$

dengan A sebagai simbol non-terminal, sedangkan α merupakan rangkaian simbol terminal maupun non-terminal.

Sebagai contoh, aturan assignment dapat ditulis sebagai:

`<assignment-statement> -> <variable> becomes <expression>`

Untuk input `a := 5`, Parse treenya dapat direpresentasikan sebagai berikut :

```
<assignment-statement>
├── <variable>
│   └── ident(a)
├── becomes
└── <expression>
    ├── <simple-expression>
    │   └── <term>
    │       └── <factor>
    │           └── intcon(5)
```

Pada proyek ini, output yang dihasilkan adalah **Parse Tree**, bukan **Abstract Syntax Tree** atau AST. Parse Tree masih mempertahankan struktur grammar secara lengkap, termasuk terminal, non-terminal, dan urutan produksinya. Hal ini membuat parse tree cocok digunakan untuk menunjukkan bahwa token-token hasil lexer benar-benar mengikuti grammar bahasa Arion.

Parse tree berguna untuk:

1. Memvisualisasikan struktur sintaks program.
2. Menunjukkan proses penurunan input dari grammar.
3. Membantu debugging parser.
4. Menjadi dasar untuk tahap berikutnya, seperti semantic analysis.

Dalam implementasi program, parse tree dapat direpresentasikan menggunakan struktur node, misalnya `ParseNode`, yang memiliki `label` dan daftar `children`.

C. Recursive Descent Parser

Recursive Descent Parser adalah metode parsing top-down, yaitu proses parsing dimulai dari simbol awal grammar, kemudian menurunkan aturan-aturan grammar hingga sesuai dengan token input. Pada metode ini, setiap non-terminal pada grammar diimplementasikan sebagai fungsi.

Sebagai contoh:

```
<program>          -> parseProgram()  
<statement>        -> parseStatement()  
<expression>       -> parseExpression()  
<compound-statement> -> parseCompoundStatement()
```

Misalnya terdapat aturan grammar:

```
<compound-statement> -> beginsy <statement-list> endsy
```

Maka aturan tersebut dapat diimplementasikan ke dalam fungsi parser dengan alur:

```
parseCompoundStatement():  
cocokkan beginsy  
parseStatementList()  
cocokkan endsy
```

Artinya, fungsi `parseCompoundStatement()` akan memastikan bahwa sebuah compound statement diawali oleh token `beginsy`, kemudian berisi `statement-list`, dan ditutup oleh token `endsy`.

Keunggulan Recursive Descent Parser adalah:

1. Struktur implementasi dekat dengan bentuk grammar.
2. Setiap non-terminal dapat dibuat sebagai fungsi terpisah.
3. Program lebih mudah dibaca dan dipelihara.
4. Error handling lebih mudah dilakukan karena kesalahan dapat dilacak dari fungsi grammar yang sedang berjalan.
5. Cocok digunakan untuk parser bahasa Arion pada milestone ini.

Namun, Recursive Descent Parser kurang cocok untuk grammar yang memiliki left recursion. Contoh left recursion:

```
<expression> -> <expression> plus <term> | <term>
```

Aturan seperti ini dapat menyebabkan fungsi parser memanggil dirinya sendiri tanpa henti. Oleh karena itu, grammar perlu ditulis ulang agar lebih sesuai untuk parsing top-down, misalnya menjadi:

```
<expression> -> <term> (plus <term>)*
```

Dengan pendekatan Recursive Descent Parser, parser bahasa Arion dapat dibangun secara modular, karena setiap bagian grammar seperti program, declaration, statement, expression, dan factor dapat diproses oleh fungsi parser masing-masing.

Perancangan & Implementasi

A. Arsitektur Program

Program pada milestone ini dirancang sebagai kelanjutan dari hasil milestone sebelumnya. Secara umum, alur program terdiri dari dua tahap utama, yaitu proses lexical analysis dan syntax analysis.

Alur program dapat digambarkan sebagai berikut

Source Code → Lexer → Token Stream → Parser → Parse Tree

Dengan alur tersebut, lexer bertugas membaca source code dan menghasilkan daftar token, sedangkan parser bertugas membaca token tersebut untuk membangun parse tree.

Komponen utama dalam program adalah sebagai berikut.

1) **token.h**

Berisi definisi jenis token melalui `TokenType`, struktur data `Token`, serta fungsi bantuan seperti `tokenTypeToString()` untuk mengubah jenis token menjadi bentuk string.

2) **lexer.cpp** dan **lexer.h**

Bertanggung jawab melakukan tokenisasi source code. Hasil dari lexer berupa daftar token yang akan menjadi input bagi parser.

3) **parser.cpp** dan **parser.h**

Berisi implementasi parser dengan metode Recursive Descent. File ini memuat fungsi-fungsi parsing untuk setiap aturan grammar bahasa Arion.

4) **parse_tree.h** dan **parse_tree.cpp**

Berisi struktur data `ParseNode` yang digunakan untuk membangun parse tree, serta fungsi untuk mencetak parse tree ke terminal atau file output.

Dengan pembagian tersebut, program menjadi lebih modular karena setiap bagian memiliki tanggung jawab yang jelas. Lexer hanya berfokus pada pembentukan token, parser berfokus pada analisis sintaks, dan parse tree berfokus pada representasi hasil parsing.

B. Aturan Grammar

a. Grammar Program

```
<program> → <program-header> <declaration-part>  
<compound-statement> period  
  
<program-header> → programsy ident semicolon  
  
<block> → <declaration-part> <compound-statement>
```

b. Grammar Deklarasi

```
<declaration-part> → <const-declaration>* <type-declaration>*  
<var-declaration>* <subprogram-declaration>*  
  
<const-declaration> → constsy (ident eql <constant>  
semicolon)+  
  
<constant> → charcon | string | (plus | minus)? (ident | intcon  
| realcon)  
  
<type-declaration> → typesy (ident eql <type> semicolon)+  
  
<var-declaration> → varsy (<identifier-list> colon <type>  
semicolon)+  
  
<identifier-list> → ident (comma ident)*
```

c. Grammar Tipe Data

```
<type> → ident | <array-type> | <range> | <enumerated> |  
<record-type>  
  
<array-type> → arraysy lbrack (<range> | ident) rbrack ofsy  
<type>  
  
<range> → <constant> period period <constant>  
  
<enumerated> → lparent ident (comma ident)* rparent  
  
<record-type> → recordsy <field-list> endsy  
  
<field-list> → <field-part> (semicolon <field-part>)*  
  
<field-part> → <identifier-list> colon <type>
```


d. Grammar Subprogram

```
<subprogram-declaration> → <procedure-declaration> |  
<function-declaration>  
  
<procedure-declaration> → proceduresy ident  
<formal-parameter-list>? semicolon <block> semicolon  
  
<function-declaration> → functionsy ident  
<formal-parameter-list>? colon ident semicolon <block>  
semicolon  
  
<formal-parameter-list> → lparent <parameter-group> (semicolon  
<parameter-group>)* rparent  
  
<parameter-group> → <identifier-list> colon (ident |  
<array-type>)
```

e. Grammar Statement Dasar dan Compound

```
<compound-statement> → beginsy <statement-list> endsy  
  
<statement-list> → <statement> (semicolon <statement>)*  
  
<statement> → (<assignment-statement> | <if-statement> |  
<case-statement> | <while-statement> | <repeat-statement> |  
<for-statement> | <procedure/function-call>)?
```

```
<variable> → ident <component-variable>*  
  
<component-variable> → lbrack <index-list> rbrack | period  
ident  
  
<index-list> → (intcon | charcon | ident) (comma  
<index-list>)*
```

```
<assignment-statement> → <variable> becomes <expression>  
  
<procedure/function-call> → ident (lparent <parameter-list>?  
rparent) ?
```

`<parameter-list> → <expression> (comma <expression>)*`

f. Grammar Statement Control

`<if-statement> → ifsy <expression> thensy <statement> (elsesy <statement>)?`

`<case-statement> → casesy <expression> ofsy <case-block> endsy`

`<case-block> → <constant> (comma <constant>)* colon
<statement> (semicolon <case-block>?)*`

`<while-statement> → whilesy <expression> dosy
<compound-statement>`

`<repeat-statement> → repeatsy <statement-list> untilsy
<expression>`

`<for-statement> → forsy ident becomes <expression> (tosy |
downtosy) <expression> dosy <compound-statement>`

Note: Production rule untuk `<while-statement>` disesuaikan dengan `<for-statement>`, **semicolon** setelah `<compound-statement>` dihilangkan.

g. Grammar Ekspresi

`<expression> → <simple-expression> (<relational-operator>
<simple-expression>)?`

`<simple-expression> → (plus | minus)? <term>
(<additive-operator> <term>)*`

`<term> → <factor> (<multiplicative-operator> <factor>)*`

`<factor> → ident | intcon | realcon | charcon | string |
lparent <expression> rparent | notsy <factor> |
<procedure/function-call> | <variable>`

`<relational-operator> → eql | neq | gtr | geq | lss | leq`

```
<additive-operator> → plus | minus | orsy
```

```
<multiplicative-operator> → times | rdiv | idiv | imod | andsy
```

C. Pemetaan Grammar ke Fungsi Parser

Setelah grammar ditentukan, setiap non-terminal pada grammar diimplementasikan sebagai fungsi parser yang terpisah. Pendekatan ini membuat struktur program lebih mudah diikuti karena nama fungsi langsung merepresentasikan rule grammar yang diproses.

Beberapa contoh pemetaan grammar ke fungsi parser adalah:

1. `<program>` diproses oleh `parseProgram()`.
2. `<program-header>` diproses oleh `parseProgramHeader()`.
3. `<declaration-part>` diproses oleh `parseDeclarationPart()`.
4. `<compound-statement>` diproses oleh `parseCompoundStatement()`.
5. `<statement-list>` diproses oleh `parseStatementList()`.
6. `<statement>` diproses oleh `parseStatement()`.
7. `<expression>` diproses oleh `parseExpression()`.

Sebagai contoh, grammar utama program:

```
<program> → <program-header> <declaration-part>  
<compound-statement> period
```

Diimplementasikan dengan alur :

```
parseProgram() :  
    parseProgramHeader()  
    parseDeclarationPart()  
    parseCompoundStatement()  
    consume(period)
```

Contoh lain, rule pada bagian statement:

```
<compound-statement> → beginsy <statement-list> endsy
```

Diimplementasikan dengan alur:

```
parseCompoundStatement() :  
    consume(beginsy)
```

```
parseStatementList()  
consume(endsy)
```

Sementara itu, rule statement umum:

```
<statement> → (<assignment-statement> | <if-statement> |  
<case-statement> | <while-statement> | <repeat-statement> |  
<for-statement> | <procedure/function-call>)?
```

diimplementasikan melalui fungsi `parseStatement()`. Fungsi ini berperan untuk menentukan jenis statement yang sedang dibaca berdasarkan token awal. Jika token awal berupa `ident`, parser akan memeriksa token berikutnya untuk membedakan apakah statement tersebut merupakan assignment, variable, atau procedure/function call. Jika token awal berupa `ifsy`, `whilesy`, `forsy`, `repeatsy`, atau `casesy`, parser akan langsung memanggil fungsi parser yang sesuai.

Bagian ekspresi juga dipisahkan menjadi beberapa fungsi agar urutan prioritas operator tetap sesuai grammar. Ekspresi diproses melalui fungsi `parseExpression()`, kemudian dilanjutkan ke `parseSimpleExpression()`, `parseTerm()`, dan `parseFactor()`.

Secara sederhana, alurnya adalah:

```
parseExpression()  
parseSimpleExpression()  
parseTerm()  
parseFactor()
```

Pembagian ini dilakukan agar operator dengan prioritas lebih tinggi, seperti perkalian dan pembagian, dapat diproses lebih dahulu dibandingkan operator penjumlahan, pengurangan, maupun operator relasional.

D. Mekanisme Kerja Parser

Parser diimplementasikan dalam bentuk class `Parser` yang menerima input berupa daftar token hasil lexer. Daftar token tersebut disimpan dalam `tokens_`, sedangkan posisi token yang sedang diproses disimpan dalam `pos_`.

Untuk membantu proses pembacaan token, parser menggunakan beberapa fungsi pendukung, yaitu:

1. `current()` untuk mengambil token yang sedang dibaca.
2. `peek()` untuk melihat token berikutnya tanpa memajukan posisi.
3. `check()` untuk memeriksa tipe token saat ini.

4. `match()` untuk mencocokkan token dan memajukan posisi jika sesuai.
5. `advance()` untuk berpindah ke token berikutnya.
6. `consume()` untuk memastikan token tertentu wajib muncul.

Fungsi `consume()` digunakan pada bagian grammar yang bersifat wajib. Jika token yang ditemukan tidak sesuai dengan token yang diharapkan, parser akan menghasilkan syntax error. Dengan mekanisme ini, setiap fungsi parser dapat memproses grammar secara berurutan dan tetap menjaga validitas struktur program.

E. Pembentukan Parse Tree dan Output

Selama proses parsing, setiap fungsi parser akan membentuk node bertipe `ParseNode`. Node ini merepresentasikan bagian grammar yang sedang diproses, misalnya `<program>`, `<declaration-part>`, `<statement>`, atau `<expression>`.

Setiap node dapat memiliki child yang berasal dari:

1. node non-terminal lain, yaitu hasil pemanggilan fungsi parser berikutnya;
2. node terminal, yaitu token aktual seperti `ident`, `intcon`, `beginsy`, atau `semicolon`.

Untuk token terminal, parser menggunakan fungsi bantuan `makeTokenNode()`. Dengan fungsi ini, token dapat langsung dimasukkan sebagai daun pada parse tree.

Contoh bentuk output parse tree:

```
<program>
├── <program-header>
│   ├── programsy
│   ├── ident(Hello)
│   └── semicolon
├── <declaration-part>
├── <compound-statement>
└── period
```

Setelah proses parsing selesai, parse tree diubah menjadi bentuk teks menggunakan `renderParseTree()`. Hasil tersebut kemudian dapat ditampilkan ke terminal dan disimpan ke file output. Dengan demikian, program tidak hanya memeriksa validitas sintaks, tetapi juga menghasilkan representasi struktur program dalam bentuk parse tree.

Pengujian

Untuk memastikan bahwa syntax analyzer dapat mengenali seluruh komponen bahasa Arion dengan benar, dilakukan serangkaian pengujian menggunakan berbagai skenario input yang menghasilkan output.

Pengujian 1

Diberikan input sebagai berikut:

```
program Hello;
var
  a, b: integer;
begin
  a := 5;
  b := a + 10;
  writeln('Result = ', b)
end.
```

Dihasilkan output sebagai berikut:

```
<program>
|-- <program-header>
|   |-- programsy
|   |-- ident(Hello)
|   \-- semicolon
|-- <declaration-part>
|   \-- <var-declaration>
|       |-- varsy
|       |-- <identifier-list>
|           |-- ident(a)
|           |-- comma
|           \-- ident(b)
|       |-- colon
|       |-- <type>
|           \-- ident(integer)
|       \-- semicolon
|-- <compound-statement>
|   |-- beginsy
|   |-- <statement-list>
|       |-- <statement>
|           \-- <assignment-statement>
|               |-- <variable>
|                   \-- ident(a)
|               |-- becomes
|               \-- <expression>
|                   \-- <simple-expression>
|                       \-- <term>
|                           \-- <factor>
```

```

|-- intcon(5)
|-- semicolon
|-- <statement>
|-- <assignment-statement>
|-- <variable>
|-- ident(b)
|-- becomes
|-- <expression>
|-- <simple-expression>
|-- <term>
|-- <factor>
|-- ident(a)
|-- <additive-operator>
|-- plus
|-- <term>
|-- <factor>
|-- intcon(10)
|-- semicolon
|-- <statement>
|-- <procedure/function-call>
|-- ident(writeln)
|-- lparent
|-- <parameter-list>
|-- <expression>
|-- <simple-expression>
|-- <term>
|-- <factor>
|-- string('Result = ')
|-- comma
|-- <expression>
|-- <simple-expression>
|-- <term>
|-- <factor>
|-- ident(b)
|-- rparent
|-- endsy
|-- period

```

Pengujian 2

Diberikan input sebagai berikut:

```

program TestKontrol;
var
  x, i: integer;
begin
  if x > 0 then
    x := x - 1

```



```

else
  x := 0;
while x > 0 do
begin
  x := x - 1
end;
for i := 1 to 5 do
begin
  x := x + i
end
end.

```

Dihasilkan output sebagai berikut:

```

<program>
|-- <program-header>
|   |-- programsy
|   |-- ident(TestKontrol)
|   \-- semicolon
|-- <declaration-part>
|   \-- <var-declaration>
|       |-- varsy
|       |-- <identifier-list>
|           |-- ident(x)
|           |-- comma
|           \-- ident(i)
|       |-- colon
|       |-- <type>
|           \-- ident(integer)
|       \-- semicolon
|-- <compound-statement>
|   |-- beginsy
|   |-- <statement-list>
|       |-- <statement>
|           \-- <if-statement>
|               |-- ifsy
|               |-- <expression>
|                   |-- <simple-expression>
|                       \-- <term>
|                           \-- <factor>
|                               \-- ident(x)
|                   |-- <relational-operator>
|                       \-- gtr
|                   \-- <simple-expression>
|                       \-- <term>
|                           \-- <factor>
|                               \-- intcon(0)
|               |-- thensy
|               \-- <statement>

```

```

|-- <assignment-statement>
|   |-- <variable>
|   |   |-- ident(x)
|   |-- becomes
|   |-- <expression>
|       |-- <simple-expression>
|           |-- <term>
|           |   |-- <factor>
|           |   |   |-- ident(x)
|           |-- <additive-operator>
|           |   |-- minus
|           |-- <term>
|           |   |-- <factor>
|           |   |   |-- intcon(1)
|-- elsey
|-- <statement>
|   |-- <assignment-statement>
|       |-- <variable>
|       |   |-- ident(x)
|       |-- becomes
|       |-- <expression>
|           |-- <simple-expression>
|               |-- <term>
|               |   |-- <factor>
|               |   |   |-- intcon(0)
-- semicolon
-- <statement>
|   |-- <while-statement>
|       |-- whilesy
|       |-- <expression>
|           |-- <simple-expression>
|               |-- <term>
|               |   |-- <factor>
|               |   |   |-- ident(x)
|           |-- <relational-operator>
|           |   |-- gtr
|       |-- <simple-expression>
|           |-- <term>
|           |   |-- <factor>
|           |   |   |-- intcon(0)
|-- dosy
|-- <compound-statement>
|   |-- beginsy
|   |-- <statement-list>
|       |-- <statement>
|           |-- <assignment-statement>
|               |-- <variable>
|               |   |-- ident(x)
|               |-- becomes

```

```

|-- <expression>
|-- <simple-expression>
|-- <term>
|-- <factor>
|-- ident(x)
|-- <additive-operator>
|-- minus
|-- <term>
|-- <factor>
|-- intcon(1)
|-- endsy
|-- semicolon
|-- <statement>
|-- <for-statement>
|-- forsy
|-- ident(i)
|-- becomes
|-- <expression>
|-- <simple-expression>
|-- <term>
|-- <factor>
|-- intcon(1)
|-- tosy
|-- <expression>
|-- <simple-expression>
|-- <term>
|-- <factor>
|-- intcon(5)
|-- dosy
|-- <compound-statement>
|-- beginsy
|-- <statement-list>
|-- <statement>
|-- <assignment-statement>
|-- <variable>
|-- ident(x)
|-- becomes
|-- <expression>
|-- <simple-expression>
|-- <term>
|-- <factor>
|-- ident(x)
|-- <additive-operator>
|-- plus
|-- <term>
|-- <factor>
|-- ident(i)
|-- endsy
|-- endsy

```

```
\-- period
```

Pengujian 3

Diberikan input sebagai berikut:

```
program TestSubprogram;
var
    hasil: integer;
procedure sapa(n: integer);
begin
    writeln(n)
end;
function tambah(a, b: integer): integer;
begin
    tambah := a + b
end;
begin
    sapa(1);
    hasil := tambah(3, 4)
end.
```

Dihasilkan output sebagai berikut:

```
<program>
|-- <program-header>
|   |-- programsy
|   |-- ident(TestSubprogram)
|   \-- semicolon
|-- <declaration-part>
|   |-- <var-declaration>
|   |   |-- varsy
|   |   |-- <identifier-list>
|   |   |   |-- ident(hasil)
|   |   |-- colon
|   |   |-- <type>
|   |   |   |-- ident(integer)
|   |   \-- semicolon
|   |-- <subprogram-declaration>
|   |   \-- <procedure-declaration>
|   |       |-- proceduresy
|   |       |-- ident(sapa)
|   |       |-- <formal-parameter-list>
|   |       |   |-- lparent
|   |       |   |-- <parameter-group>
|   |       |   |   |-- <identifier-list>
|   |       |   |   |   |-- ident(n)
```

```

| | | |-- colon
| | | \-- ident(integer)
| | | \-- rparent
|-- semicolon
|-- <block>
| | |-- <declaration-part>
| | | \-- <compound-statement>
| | | |-- beginsy
| | | |-- <statement-list>
| | | | | \-- <statement>
| | | | | | \-- <procedure/function-call>
| | | | | | |-- ident(writeln)
| | | | | | |-- lparent
| | | | | | |-- <parameter-list>
| | | | | | | | \-- <expression>
| | | | | | | | | \-- <simple-expression>
| | | | | | | | | | \-- <term>
| | | | | | | | | | | \-- <factor>
| | | | | | | | | | | \-- ident(n)
| | | | | | \-- rparent
| | | | \-- endsy
| | | \-- semicolon
|-- <subprogram-declaration>
|-- <function-declaration>
| | |-- functionsy
| | |-- ident(tambah)
| | |-- <formal-parameter-list>
| | | |-- lparent
| | | |-- <parameter-group>
| | | | | \-- <identifier-list>
| | | | | | \-- ident(a)
| | | | | | \-- comma
| | | | | | \-- ident(b)
| | | | | \-- colon
| | | | \-- ident(integer)
| | | \-- rparent
|-- colon
|-- ident(integer)
|-- semicolon
|-- <block>
| | |-- <declaration-part>
| | | \-- <compound-statement>
| | | |-- beginsy
| | | |-- <statement-list>
| | | | | \-- <statement>
| | | | | | \-- <assignment-statement>
| | | | | | |-- <variable>
| | | | | | | \-- ident(tambah)
| | | | | | \-- becomes

```

```

|-- <expression>
|-- <simple-expression>
|-- <term>
|-- <factor>
|-- ident(a)
|-- <additive-operator>
|-- plus
|-- <term>
|-- <factor>
|-- ident(b)
|-- endsy
|-- semicolon
-- <compound-statement>
-- beginsy
-- <statement-list>
-- <statement>
-- <procedure/function-call>
-- ident(sapa)
-- lparent
-- <parameter-list>
-- <expression>
-- <simple-expression>
-- <term>
-- <factor>
-- intcon(1)
-- rparent
-- semicolon
-- <statement>
-- <assignment-statement>
-- <variable>
-- ident(hasil)
-- becomes
-- <expression>
-- <simple-expression>
-- <term>
-- <factor>
-- <procedure/function-call>
-- ident(tambah)
-- lparent
-- <parameter-list>
-- <expression>
-- <simple-expression>
-- <term>
-- <factor>
-- intcon(3)
-- comma
-- <expression>
-- <simple-expression>
-- <term>

```

[illegible]

Pengujian 4

Diberikan input sebagai berikut:

```
program TestDeklarasi;
const
    MAX == 100;
type
    Titik == record
        x: integer;
        y: integer
    end;
var
    p: Titik;
begin
    p.x := 10;
    p.y := 20
end.
```

Dihasilkan output sebagai berikut:

```
<program>
|-- <program-header>
|   |-- programsy
|   |-- ident(TestDeklarasi)
|   \-- semicolon
|-- <declaration-part>
|   |-- <const-declaration>
|   |   |-- constsy
|   |   |-- ident(MAX)
|   |   |-- eql
|   |   |-- <constant>
|   |   |   \-- intcon(100)
|   |   \-- semicolon
|   |-- <type-declaration>
|   |   |-- typesy
|   |   |-- ident(Titik)
|   |   |-- eql
|   |   |-- <type>
|   |   |   \-- <record-type>
|   |   |       |-- recordsy
|   |   |       \-- <field-list>
```

```

|-- <field-part>
|   |-- <identifier-list>
|   |   |-- ident(x)
|   |   |-- colon
|   |   |-- <type>
|   |   |   |-- ident(integer)
|   |-- semicolon
|-- <field-part>
|   |-- <identifier-list>
|   |   |-- ident(y)
|   |   |-- colon
|   |   |-- <type>
|   |   |   |-- ident(integer)
|-- endsy
|-- semicolon
|-- <var-declaration>
|   |-- varsy
|   |-- <identifier-list>
|   |   |-- ident(p)
|   |-- colon
|   |-- <type>
|   |   |-- ident(Titik)
|-- semicolon
-- <compound-statement>
|   |-- beginsy
|   |-- <statement-list>
|   |   |-- <statement>
|   |   |   |-- <assignment-statement>
|   |   |   |   |-- <variable>
|   |   |   |   |   |-- ident(p)
|   |   |   |   |   |-- <component-variable>
|   |   |   |   |   |   |-- period
|   |   |   |   |   |   |-- ident(x)
|   |   |   |   |-- becomes
|   |   |   |-- <expression>
|   |   |   |   |-- <simple-expression>
|   |   |   |   |   |-- <term>
|   |   |   |   |   |   |-- <factor>
|   |   |   |   |   |   |-- intcon(10)
|   |   |-- semicolon
|-- <statement>
|   |-- <assignment-statement>
|   |   |-- <variable>
|   |   |   |-- ident(p)
|   |   |   |-- <component-variable>
|   |   |   |   |-- period
|   |   |   |   |-- ident(y)
|   |   |-- becomes
|-- <expression>

```



```
| | | \-- <simple-expression>  
| | |   \-- <term>  
| | |     \-- <factor>  
| | |       \-- intcon(20)  
| | \-- endsy  
\-- period
```

Pengujian 5

Diberikan input sebagai berikut:

```
program TestError;
var
  x: integer;
const
  MAX == 10;
begin
  x := MAX
end.
```

Dihasilkan output sebagai berikut:

```
[PARSER] Syntax error at line 4: declarations must be ordered as
const, type, var, then subprogram
```

Kesimpulan dan Saran

Berdasarkan hasil pengembangan dan pengujian yang telah dilakukan pada milestone ini, bagian ini merangkum pencapaian fungsionalitas program serta memberikan rekomendasi untuk optimasi pada milestone pengembangan selanjutnya.

Kesimpulan

Tahap syntax analysis merupakan tahap kedua dalam proses kompilasi yang berfungsi memeriksa kebenaran struktur sintaks dari urutan token yang dihasilkan oleh lexer. Berdasarkan perancangan dan implementasi yang telah dilakukan, pembangunan parser untuk bahasa Arion berhasil diimplementasikan menggunakan algoritma Recursive Descent dengan bahasa C++.

Parser berhasil membangun Parse Tree yang merepresentasikan hierarki struktur program sesuai dengan grammar yang telah didefinisikan. Setiap non-terminal pada grammar dipetakan menjadi fungsi parser yang terpisah, sehingga program menjadi modular dan mudah dipelihara. Dari hasil pengujian yang dilakukan terhadap 5 kasus uji yang mencakup berbagai fitur bahasa Arion, termasuk deklarasi variabel, statement kontrol, subprogram, tipe record, dan penanganan error, di mana seluruh kasus uji berhasil menghasilkan output yang sesuai dengan grammar.

Program juga telah dilengkapi dengan mekanisme error handling yang baik. Ketika terdapat kesalahan sintaks, program memberikan pesan error yang informatif yang mencantumkan nomor baris dan deskripsi kesalahan tanpa menyebabkan crash.

Saran

Terlepas dari program yang berjalan dengan lancar, terdapat beberapa saran yang dibagi menjadi tiga koridor: kepada kelompok ISL, pembuat spesifikasi Tugas Besar, dan pengembang selanjutnya. Saran-saran tersebut adalah sebagai berikut.

1. Kelompok ISL

Mekanisme error recovery dapat ditingkatkan agar parser tidak langsung berhenti saat menemukan satu kesalahan, melainkan mencoba melakukan sinkronisasi dan melanjutkan parsing untuk melaporkan sebanyak mungkin kesalahan sintaks dalam satu kali eksekusi. Selain itu, informasi lokasi error dapat diperkaya dengan menambahkan nomor kolom selain nomor baris.

2. Pembuat Spesifikasi

Contoh output Parse Tree yang diberikan dalam spesifikasi dapat dilengkapi agar mencakup seluruh node yang didefinisikan dalam grammar, seperti node <statement>, <variable>, dan <additive-operator>. Hal ini akan mengurangi ambiguitas interpretasi antar kelompok.

3. Pengembang Selanjutnya

Sangat disarankan agar pengembangan lanjutan ke tahap semantic analysis tetap mempertahankan struktur Parse Tree yang telah dihasilkan sebagai input. Konversi Parse Tree ke Abstract Syntax Tree (AST) perlu dilakukan dengan hati-hati agar informasi yang dibutuhkan untuk analisis semantik tidak hilang.

Lampiran

Link Github: <https://github.com/raulinn/ISL-Tubes-IF2224-2026>

Pembagian Tugas:

NIM	Nama	Bagian	Persentase Keseluruhan
13524044	Narendra Dharma Wistara M	Program, deklarasi (const, type, var), tipe data (array, range, enumerated, record)	25%
13524060	Steven Tan	Ekspresi (expression, simple-expression, term, factor, operator relational/aditif/multiplikatif)	25%
13524090	Nashiruddin Akram	Statement dasar & compound (compound-statement, statement-list, assignment, procedure/function-call)	25%
13524100	Arghawisesa Dwinanda Arham	Subprogram (procedure, function), statement kontrol (if, case, while, repeat, for)	25%

Referensi

- [1] J. E. Hopcroft, R. Motwani, and J. D. Ullman, *Introduction to automata theory, languages, and computation*, 3. ed. Boston San Francisco Munich: Pearson, 2007.
- [2] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, Eds., *Compilers: principles, techniques, & tools*, 2. ed., Pearson internat. ed. Boston Munich: Pearson Addison-Wesley, 2007
- [3] Laboratorium Ilmu Rekayasa dan Komputasi, "Spesifikasi Milestone 2 - Tubes IF2224 TBFO", Institut Teknologi Bandung, Bandung, 2026. [Online]. Available: [Spesifikasi Milestone 2 - Tubes IF2224 TBFO](#)